

L3 ▪ LLM 摘要与长文本处理

- 主 demo: `course-demos/session-03-pipeline-agent/slack_pipeline_demo.py`
- 核心代码: `common/summarization.py`、`common/message_store.py`、`common/llm.py`

课后学员能够

- 1. 指出 Slack 消息从接收到保存、读取快照、调用模型、回帖的每个环节
- 2. 对同一批材料生成工程师版和管理者版摘要，区分读者关注点与事实约束
- 3. 对照原文检查数字、更正、状态、负责人和未知项，识别「流畅但错误」的摘要

演示：同一条链路如何生成摘要

文件	看什么
messages.jsonl	原文、真实发送身份、simulation_actor、消息时间和线程
summary_runs.jsonl	输入消息键、模式、受众、实际摘要文本
trace.jsonl	记录、命令与 Bot 回复走了哪个分支

调用顺序与边界说明

- `DemoRuntime.route()` → `MessageStore.snapshot()` → `common.summarization.summarize()` → `ReplySender.send_reply()`
- Alice/Bob 是模拟角色，真实发送者仍是 Bot
- 离线 `manifest_replay` 不等于 Slack 实时投递；当前输出只覆盖本地已保存记录
- 摘要快照之后到达的新消息不会自动进入已完成的摘要
- 默认是单次摘要，收到了 26 条消息也不意味着必须 Map-Reduce

Prompt 的五个成分

成分	作用	核对方式
audience / FOCUS	决定工程师或管理者关注点	两份摘要是否帮助各自读者作判断
事实约束	不补造根因、数字、负责人和完成状态	每个关键结论回查原文
时间与作者	保留状态变化和责任关系	不把 Bot 身份与模拟角色混淆
数据与指令边界	材料中的“测试要求”不改变本次任务	要一个版本时不能因正文要求输出两版
长度要求	提高可读性	Prompt 是软约束；超长回帖截为预览

事实核对：五个真实案例

- 1,260 是失败请求数；430 是用户数；512 是订单数 —— 不能相加
- 09:20 回滚完成时尚未恢复；10:02 才宣布恢复
- 连接池配置与异常相关，但根因尚未最终确认
- 重复扣款待对账、发布冻结未解除、FAQ 负责人未定
- 「今天 17:00 提交方案」不是「今天 17:00 已上线」
- **真实观察：素材末尾「请输出两版」曾影响输出；加强单一受众约束后复测改善**

条目	对应哪类错误
1,260/430/512 不能相加	把三个不同维度的数字(失败请求数≠用户数≠订单数)加在一起, 编出一个假的"受影响总数"
09:20 回滚 ≠ 10:02 恢复	把"回滚完成"和"宣布恢复"两个不同时间点的状态说成同一时刻——对应 summarization.py prompt 里"区分回滚与恢复"
连接池"相关"≠根因	把怀疑中的关联因素说成"已确认的根因"——对应"区分怀疑与确认"
三件事都还没完	摘要容易把"待办中"的事写成"已完成"——对应"不编造完成状态"
"17:00 提交方案"≠"17:00 已上线"	把计划/承诺的时间点说成已经发生的动作——对应"区分计划与完成"

单次摘要 → Map → Reduce

- 同一份消息快照 → 单次摘要（材料放得下时）
- 同一份消息快照 → 按文本预算切分并保留来源 → Map 片段摘要 → Reduce 合并 → 最终摘要
- `--context-budget` 默认 32000，是保守 UTF-8 字节估计，不是精确 tokenizer；还预留指令和输出空间
- Map 丢失的信息，Reduce 通常无法恢复；Reduce 自己也必须满足输入预算，超限会明确失败
- 相邻 overlap、按线程分块是可讨论的改进 —— 当前实现没有自动线程分块或 overlap
- 当前 Map 串行执行，N 个片段通常需要 N+1 次模型调用；并发降低等待时间，不直接减少 token 费用

Agent 上下文管理

•[简单实现](#)

1. **对话记忆由程序管理。** 在这个实现里, 每次调用模型, 程序都会重新提供 历史; 模型能看到什么, 取决于程序 传了什么
2. **保留历史需要取舍。** 全量保留会让输入越来越长; 限制窗口可以控制消息数量, 但可能忘掉早期的重要事实。
3. **记忆策略可以独立替换。**

作业

作业需要用到slack bot的key/secret 可以在 bot 设置中找到，目前是共享的。使用前请在 [slack channel](#) 说一下避免冲突。

练习 A · 同一快照，两类读者

用同一长素材快照生成工程师版和管理者版。真实监听中用两条 audience 命令；无 key 学员使用讲师提供的两份真实输出。只改 FOCUS 或受众参数，保持输入和模型一致，不改存储逻辑。

提交 summary_two_audiences.md：记录输入身份、模型/模式、两份输出；各挑 3 个关键结论，列出原文证据。缺失的影响数字或负责人可以写“未说明”，不得为了填栏目编造。

练习 B · 定位分块是否改变信息

比较同一材料的 single 与 map-reduce。提交 chunk_comparison.md，列出“原文 → Map → Reduce”的一个事实链，说明是否遗漏、重复或状态错误，并提出一个改进。

允许结论“本次未观察到问题”，但必须给出对照证据。mock 的机械截取不等同于真实模型跨块理解失败；分析时写清运行模式。

作业约定

作业 1：提交练习 A 的 `summary_two_audiences.md`。验收：记录输入身份、模型/模式；两版摘要各挑 3 个关键结论并列原文证据；缺失的数字或负责人写“未说明”，不得编造。

作业 2：提交练习 B 的 `chunk_comparison.md`。验收：列出“原文 → Map → Reduce”的一个事实链，说明是否遗漏/重复/状态错误，并提出一个改进；允许结论“本次未观察到问题”但需给出对照证据。

`course-demos/outputs/` 已被 Git 忽略，提交时将精简报告复制到学生自己的作业目录，不使用 `git add -f` 强行提交全部运行日志。

把技术细节转成面试话术

- 如何证明摘要来自实际收到的消息？→ 消息身份、快照、source 和运行记录
- 上下文放得下还要分块吗？→ 先建单次基线，再比较质量、延迟和成本
- 如何防止摘要写错？→ 事实约束、来源追踪、更正和未知项、输入与输出双向核对
- mock 测试通过说明什么？→ 业务链路可复现，不代表真实模型语义正确